

Algoritmi e Strutture Dati

Capitolo 1: Un'introduzione informale agli algoritmi Spiegazione Analitica e Formale dei Concetti d'Esame

Tratto da: C. Demetrescu, I. Finocchi, G. F. Italiano (McGraw-Hill)

Indice

1	Concetti Fondamentali e Definizioni	3
1.1	Definizione Informale di Algoritmo	3
1.2	L'Esempio Pratico: L'Algoritmo <code>preparaCaffè</code>	3
1.3	Distinzione tra Algoritmo e Programma	3
1.4	Lo Pseudocodice	4
1.5	Requisiti Fondamentali: Correttezza ed Efficienza	4
1.6	Perché Analizzare gli Algoritmi? Analisi Teorica vs Sperimentale	4
2	Il Caso di Studio: I Numeri di Fibonacci	4
2.1	L'Isola dei Conigli (Dinamica delle Popolazioni)	4
2.2	Le Regole del Modello Demografico	5
2.3	Evoluzione Temporale e Albero della Popolazione	5
3	Primo Approccio: Algoritmo Numerico (<code>fibonacci1</code>)	5
3.1	La Formula Chiusa di Binet	5
3.2	Pseudocodice dell'Algoritmo <code>fibonacci1</code>	6
3.3	Analisi della Correttezza e Limiti di Precisione	6
4	Secondo Approccio: Ricorsione Diretta (<code>fibonacci2</code>)	6
4.1	Struttura dell'Algoritmo Ricorsivo	6
4.2	Misurazione del Tempo di Esecuzione	6
4.3	Relazione di Ricorrenza per il Tempo $T(n)$	7
4.4	Analisi tramite l'Albero della Ricorsione	7
4.5	Valutazione dell'Efficienza	7
5	Terzo e Quarto Approccio: Programmazione Dinamica e Memoria	8
5.1	Diagnosi dell'Inefficienza: La Ridondanza	8
5.2	Algoritmo <code>fibonacci3</code> (Memorizzazione Tabellare)	8
5.3	Risultati Empirici e Confronto dei Tempi Reali	8
5.4	La Risorsa Spazio di Memoria	8
5.5	Algoritmo <code>fibonacci4</code> (Ottimizzazione dello Spazio)	9
6	Formalizzazione Teorica: La Notazione Asintotica O	9
6.1	Superamento del Conteggio delle Linee di Codice	9
6.2	Definizione Formale di O -Grande (Big- O)	9
6.3	Esempi di Semplificazione Asintotica	9

7	Approccio Avanzato: Calcolo Matriciale e Potenze Ricorsive	10
7.1	Possiamo fare meglio di $O(n)$?	10
7.2	Proprietà Matriciale dei Numeri di Fibonacci	10
7.3	Algoritmo <code>fibonacci5</code> (Moltiplicazione Sequenziale)	10
7.4	Il Principio dell'Elevamento a Potenza Ricorsivo	11
7.5	Algoritmo <code>fibonacci6</code>	11
7.6	Analisi della Complessità Temporale di <code>fibonacci6</code>	11
8	Quadro Sinottico Comparativo di tutti gli Algoritmi	12

1 Concetti Fondamentali e Definizioni

1.1 Definizione Informale di Algoritmo

Nel contesto dell'informatica teorica e della progettazione del software, un **algoritmo** definisce il procedimento logico risolutivo per un dato problema di calcolo.

Definizione 1.1 (Algoritmo - Definizione Informale). Un algoritmo è un insieme finito di istruzioni, definite passo per passo in modo univoco e non ambiguo, tali da poter essere eseguite meccanicamente da un esecutore (umano o artificiale) e capaci di produrre un determinato risultato in un tempo finito.

Analizzando la definizione nei suoi costituenti fondamentali:

1. **Insieme di istruzioni:** deve esistere un insieme di operazioni elementari ben specificate.
2. **Passo per passo:** l'esecuzione procede in una sequenza temporale discreta ed ordinata.
3. **Esecuzione meccanica:** l'esecutore non deve interpretare creativamente le istruzioni, né fare ricorso all'intuito. Ogni operazione deve essere deterministicamente eseguibile.
4. **Terminazione e Risultato:** per ogni input valido, la sequenza di passi deve terminare e restituire l'output corretto.

1.2 L'Esempio Pratico: L'Algoritmo preparaCaffè

Per esemplificare il concetto di procedura meccanica e sequenziale, consideriamo l'algoritmo elementare per la preparazione del caffè mediante moka:

```
algoritmo preparaCaffè
1. Svita la caffettiera.
2. Riempi d'acqua il serbatoio della caffettiera.
3. Inserisci il filtro.
4. Riempi il filtro con la polvere di caffè.
5. Avvita la parte superiore della caffettiera.
6. Metti la caffettiera, così predisposta, su un fornello acceso.
7. Spegni il fornello quando il caffè è pronto.
8. Versa il caffè nella tazzina.
```

Listing 1: Algoritmo preparaCaffè

Questo esempio mostra come un obiettivo complesso possa essere decomposto in un'ordinata successione di sotto-operazioni elementari e deterministiche.

1.3 Distinzione tra Algoritmo e Programma

È fondamentale non confondere il concetto astratto di algoritmo con la sua realizzazione concreta:

- **Algoritmo:** Rappresenta l'astrazione concettuale del procedimento solutorio. È indipendente dal linguaggio di programmazione, dall'hardware, dall'architettura del calcolatore e dal sistema operativo.
- **Programma:** È la codifica concreta di un algoritmo espressa in uno specifico linguaggio di programmazione (es. C, C++, Java, Python), direttamente eseguibile o traducibile (tramite compilatore o interprete) in istruzioni macchina.

1.4 Lo Pseudocodice

Per descrivere gli algoritmi mantenendo la massima generalità formale senza perdersi nei dettagli sintattici di un particolare linguaggio, si adotta lo **pseudocodice**.

Caratteristiche dello pseudocodice:

1. Utilizza costrutti strutturati derivati dai linguaggi di programmazione reali (**if-then-else**, cicli **for**, **while**, assegnamenti).
2. Integra frasi in linguaggio naturale (es. italiano) laddove l'operazione sia concettualmente chiara ma sintatticamente verbosa.
3. Permette una traduzione pressoché meccanica in un qualsiasi linguaggio di programmazione reale.

1.5 Requisiti Fondamentali: Correttezza ed Efficienza

La progettazione degli algoritmi si fonda su due pilastri essenziali:

1. **Correttezza**: L'algoritmo deve produrre il risultato esatto ed atteso per *ogni* istanza valida del problema in ingresso.
2. **Efficienza**: L'algoritmo deve minimizzare l'uso delle risorse di calcolo. Le due risorse critiche sono:
 - **Tempo di esecuzione**: Numero di operazioni effettuate dalla CPU.
 - **Occupazione di memoria (Spazio)**: Quantità di memoria RAM necessaria durante l'esecuzione.

1.6 Perché Analizzare gli Algoritmi? Analisi Teorica vs Sperimentale

L'analisi degli algoritmi permette di valutare le prestazioni del software prima ancora di scriverne il codice. Esistono due approcci:

Analisi Sperimentale	Analisi Teorica
Si codifica l'algoritmo e si misura il tempo di esecuzione con un cronometro/profiler.	Utilizza un modello matematico astratto del calcolatore.
Dipende fortemente dall'hardware, dal compilatore, dal sistema operativo e dal carico di lavoro.	È del tutto indipendente dalla piattaforma hardware e software.
Può essere condotta solo su un sottoinsieme limitato di input di prova.	Vale universalmente per tutte le possibili istanze di ingresso.

Tabella 1: Confronto tra analisi sperimentale e analisi teorica.

L'analisi teorica risulta quindi superiore e più affidabile per prevedere il comportamento su dati di dimensioni arbitrarie.

2 Il Caso di Studio: I Numeri di Fibonacci

2.1 L'Isola dei Conigli (Dinamica delle Popolazioni)

Nel 1202, il matematico Leonardo da Pisa (detto Fibonacci) pose il seguente problema di dinamica delle popolazioni:

«Quanto velocemente si espanderebbe una popolazione di conigli sotto appropriate condizioni? Partendo da una singola coppia di conigli in un'isola deserta, quante coppie si avrebbero nell'anno n ?»

2.2 Le Regole del Modello Demografico

Il modello biologico ideato da Fibonacci si basa su tre regole fondamentali:

1. **Tasso di riproduzione:** Ogni coppia di conigli adulti genera una nuova coppia di coniglietti (due individui) ogni anno.
2. **Maturazione sessuale:** I conigli iniziano a riprodursi soltanto al **secondo anno** dalla loro nascita (ovvero: nel primo anno sono giovani/immaturo; dal secondo anno diventano adulti e fertili).
3. **Immortalità:** I conigli non muoiono mai durante il periodo considerato.

2.3 Evoluzione Temporale e Albero della Popolazione

Esaminiamo la crescita della popolazione anno per anno:

- **Anno 1:** Viene introdotta 1 coppia giovane appena nata. Totale coppie = 1.
- **Anno 2:** La coppia diventa adulta, ma non genera ancora figli. Totale coppie = 1.
- **Anno 3:** La coppia (ora adulta) genera 1 nuova coppia giovane. Totale coppie = 2 (1 adulta, 1 giovane).
- **Anno 4:** La coppia adulta iniziale genera un'altra coppia giovane; la seconda coppia diventa adulta. Totale coppie = 3 (2 adulte, 1 giovane).

In generale, per $n \geq 3$, il numero di coppie F_n presenti nell'anno n si ottiene così:

1. Sono presenti tutte le coppie dell'anno precedente (F_{n-1}), data la regola di immortalità.
2. Ad esse si aggiungono le nuove coppie nate nell'anno n .
3. Quante nuove coppie nascono nell'anno n ? Tante quante sono le coppie fertili (adulte). Poiché i conigli diventano adulti al secondo anno, le coppie fertili nell'anno n sono esattamente pari a tutte le coppie presenti due anni prima, cioè nell'anno $n - 2$ (F_{n-2}).

Otteniamo così la celebre **relazione di ricorrenza**:

$$F_n = \begin{cases} 1 & \text{se } n = 1, 2 \\ F_{n-1} + F_{n-2} & \text{se } n \geq 3 \end{cases} \quad (1)$$

Il problema informatico che vogliamo affrontare è: **Come calcoliamo F_n in modo efficiente?**

3 Primo Approccio: Algoritmo Numerico (fibonacci1)

3.1 La Formula Chiusa di Binet

In sede di analisi matematica, è possibile dimostrare che la relazione di ricorrenza ammette una soluzione in forma chiusa (nota come *Formula di Binet*):

$$F_n = \frac{1}{\sqrt{5}} (\phi^n - \hat{\phi}^n) \quad (2)$$

dove ϕ e $\hat{\phi}$ sono le radici dell'equazione caratteristica $x^2 - x - 1 = 0$:

$$\phi = \frac{1 + \sqrt{5}}{2} \approx +1.6180339887 \quad (\text{Sezione Aurea}) \quad (3)$$

$$\hat{\phi} = \frac{1 - \sqrt{5}}{2} \approx -0.6180339887 \quad (4)$$

3.2 Pseudocodice dell'Algoritmo fibonacci1

Sfruttando direttamente la formula matematica chiusa, possiamo definire il seguente algoritmo:

```
algoritmo fibonacci1(intero n) → intero
  return 1/sqrt(5) * (phi^n - phi_hat^n)
```

Listing 2: Algoritmo fibonacci1

3.3 Analisi della Correttezza e Limiti di Precisione

L'algoritmo fibonacci1 appare teoricamente ideale, richiedendo un numero costante di operazioni algebriche. Tuttavia, **non è corretto su un calcolatore reale**.

I calcolatori rappresentano i numeri reali ($\sqrt{5}, \phi, \hat{\phi}$) mediante la rappresentazione in **virgola mobile (floating-point)** a precisione finita (es. IEEE 754). Gli errori di arrotondamento accumulati nell'elevamento a potenza si amplificano al crescere di n .

n	fibonacci1(n) (3 cifre dec.)	Arrotondamento all'intero	Valore Reale F_n
3	1.99992	2	2 (Corretto)
16	986.698	987	987 (Corretto)
18	2583.100	2583	2584 (ERRATO!)

Tabella 2: Mancanza di correttezza di fibonacci1 a causa dell'approssimazione numerica.

Conclusione esame: fibonacci1 viene scartato poiché invalido sul piano della *correttezza*.

4 Secondo Approccio: Ricorsione Diretta (fibonacci2)

4.1 Struttura dell'Algoritmo Ricorsivo

Per garantire la correttezza matematica, eliminiamo i numeri reali ed implementiamo direttamente la definizione ricorsiva lavorando unicamente con numeri interi:

```
algoritmo fibonacci2(intero n) → intero
  if (n ≤ 2) then
    return 1
  else
    return fibonacci2(n-1) + fibonacci2(n-2)
```

Listing 3: Algoritmo fibonacci2

4.2 Misurazione del Tempo di Esecuzione

Adottiamo come metrica formale per il tempo di esecuzione $T(n)$ il **numero di linee di codice mandate in esecuzione**. Si tratta di una misura astratta ed indipendente dalla piattaforma hardware.

Valutiamo l'esecuzione per piccoli valori di n :

- Se $n \leq 2$: viene eseguita 1 sola linea di codice (il controllo e il return della condizione `if`).
- Se $n = 3$: vengono eseguite 4 linee di codice:
 1. Chiamata `fibonacci2(3)`: valuta l'if (falso) e prepara la somma delle due chiamate (2 linee).
 2. Chiamata ricorsiva `fibonacci2(2)`: valuta l'if (vero) e restituisce 1 (1 linea).
 3. Chiamata ricorsiva `fibonacci2(1)`: valuta l'if (vero) e restituisce 1 (1 linea).

4.3 Relazione di Ricorrenza per il Tempo $T(n)$

In generale, per un generico $n \geq 3$, la funzione del tempo $T(n)$ soddisfa la seguente equazione di ricorrenza:

$$T(n) = \begin{cases} 1 & \text{se } n \leq 2 \\ 2 + T(n-1) + T(n-2) & \text{se } n \geq 3 \end{cases} \quad (5)$$

In un algoritmo ricorsivo, il tempo totale è pari al tempo speso nella chiamata corrente (2 linee per l'if e l'addizione) più il tempo speso in tutte le sotto-chiamate ricorsive ($T(n-1) + T(n-2)$).

4.4 Analisi tramite l'Albero della Ricorsione

Per risolvere l'equazione di ricorrenza $T(n)$, costruiamo l'**albero della ricorsione**:

- **Nodi**: Rappresentano le singole chiamate ricorsive a `fibonacci2`.
- **Archì**: Rappresentano le generazioni di sotto-chiamate.
- **Nodi Interni**: Corrispondono a chiamate con input $n \geq 3$. Ognuno di essi esegue 2 linee di codice.
- **Foglie**: Corrispondono a chiamate con input $n = 1$ oppure $n = 2$ (casi base). Ognuna di esse esegue 1 linea di codice.

Proposizione 4.1 (Conteggio esatto dei nodi e delle linee di codice). Sia dato l'albero della ricorsione generato da `fibonacci2(n)`:

1. Il numero di foglie dell'albero è esattamente pari a F_n .
2. Il numero di nodi interni di un albero binario in cui ogni nodo interno ha 2 figli è pari al numero di foglie meno uno, ovvero $F_n - 1$.
3. Il numero totale di linee di codice eseguite $T(n)$ è:

$$T(n) = 1 \cdot (\text{numero di foglie}) + 2 \cdot (\text{numero di nodi interni}) \quad (6)$$

$$T(n) = F_n + 2(F_n - 1) = 3F_n - 2 \quad (7)$$

4.5 Valutazione dell'Efficienza

Poiché la successione dei numeri di Fibonacci cresce in modo esponenziale secondo la sezione aurea $F_n \approx \frac{\phi^n}{\sqrt{5}}$, concludiamo che:

$$T(n) = 3F_n - 2 = \Theta(\phi^n) = O(2^n) \quad (8)$$

L'algoritmo `fibonacci2` ha una **complessità temporale esponenziale**. Questo lo rende estremamente lento ed inutilizzabile all'atto pratico anche per valori modesti di n (es. per $n = 58$ servono ore di calcolo, come mostrato nella Tabella 3; per $n = 100$ servirebbero millenni).

5 Terzo e Quarto Approccio: Programmazione Dinamica e Memoria

5.1 Diagnosi dell'Inefficienza: La Ridondanza

Per quale motivo `fibonacci2` è così lento? Analizzando l'albero della ricorsione, si nota una massiccia **sovrapposizione dei sottoproblemi**: l'algoritmo ricalcola continuamente da capo la soluzione per gli stessi valori. Ad esempio, per calcolare F_8 , il sotto-valore F_3 viene ricalcolato autonomamente ed inutilmente per ben 8 volte!

5.2 Algoritmo `fibonacci3` (Memorizzazione Tabellare)

L'idea fondamentale per risolvere la ridondanza consiste nell'utilizzare la **Programmazione Dinamica (approccio bottom-up)**: memorizziamo i risultati dei sottoproblemi già risolti in una struttura dati (un array):

```
algoritmo fibonacci3(intero n) → intero
  sia Fib un array di n interi
  Fib[1] ← 1
  Fib[2] ← 1
  for i = 3 to n do
    Fib[i] ← Fib[i-1] + Fib[i-2]
  return Fib[n]
```

Listing 4: Algoritmo `fibonacci3`

Complessità Temporale: Il ciclo `for` esegue $n - 2$ iterazioni. Ogni iterazione compie un'operazione elementare di somma $O(1)$. Di conseguenza, il tempo di esecuzione è **lineare**: $T(n) = O(n)$.

5.3 Risultati Empirici e Confronto dei Tempi Reali

I benefici teorici del passaggio da tempo esponenziale ad $O(n)$ si traducono in differenze abissali nei test pratici sul calcolo di F_{58} :

Architettura Hardware	<code>fibonacci2(58)</code> (Esponenziale)	<code>fibonacci3(58)</code> (Lineare)
Pentium IV 1700 MHz	15.820 sec (~ 4.4 ore)	$0.7\mu s$ (0.0000007 sec)
Pentium III 450 MHz	43.518 sec (~ 12.1 ore)	$2.4\mu s$ (0.0000024 sec)
PowerPC G4 500 MHz	58.321 sec (~ 16.2 ore)	$2.8\mu s$ (0.0000028 sec)

Tabella 3: Confronto sui tempi di esecuzione reali per il calcolo di F_{58} .

5.4 La Risorsa Spazio di Memoria

Il tempo di esecuzione non è la sola risorsa di calcolo limitata; l'**occupazione di memoria (spazio)** è ugualmente determinante.

- Se un algoritmo è lento, è possibile attendere più a lungo o utilizzare hardware più potente.
- Se un algoritmo richiede più memoria di quella fisicamente disponibile, l'esecuzione fallisce inevitabilmente (crash per *Out of Memory*).

L'algoritmo `fibonacci3` richiede un array di dimensione n , registrando un'occupazione di memoria pari ad $O(n)$ parole di memoria.

5.5 Algoritmo fibonacci4 (Ottimizzazione dello Spazio)

Osserviamo che per calcolare $Fib[i]$ non serve mantenere in memoria l'intera storia di tutti i numeri precedenti, ma sono sufficienti unicamente **gli ultimi due valori** ($Fib[i-1]$ e $Fib[i-2]$).

Possiamo quindi ridurre lo spazio allocando soltanto tre variabili scalari:

```
algoritmo fibonacci4(intero n) → intero
  a ← 1
  b ← 1
  for i = 3 to n do
    c ← a + b
    a ← b
    b ← c
  return b
```

Listing 5: Algoritmo fibonacci4

Valutazione:

- **Tempo di esecuzione:** $O(n)$ (pari a fibonacci3).
- **Occupazione di memoria:** $O(1)$ (spazio costante, indipendente da n).

6 Formalizzazione Teorica: La Notazione Asintotica O

6.1 Superamento del Conteggio delle Linee di Codice

Misurare $T(n)$ tramite il conteggio esatto delle linee di codice presenta evidenti limiti:

1. È sensibile allo stile di formattazione (andare a capo più spesso aumenta le linee sorgente senza alterare le prestazioni).
2. Rende difficile il confronto tra programmi scritti da programmatori diversi.

Serve uno strumento matematico per descrivere l'**ordine di grandezza** di $T(n)$ ignorando le costanti moltiplicative e i dettagli trascurabili.

6.2 Definizione Formale di O -Grande (Big- O)

Definizione 6.1 (Notazione Asintotica O). Siano $f(n)$ e $g(n)$ due funzioni che mappano interi positivi in reali positivi. Diciamo che:

$$f(n) = O(g(n)) \quad (9)$$

se e solo se esistono due costanti $c > 0$ ed $n_0 > 0$ tali per cui:

$$f(n) \leq c \cdot g(n) \quad \forall n \geq n_0 \quad (10)$$

Geometricamente, la funzione $g(n)$ (opportunamente scalata dalla costante c) rappresenta un **limite superiore** (*upper bound*) per $f(n)$ per tutti i valori di n sufficientemente grandi ($n \geq n_0$).

6.3 Esempi di Semplificazione Asintotica

Grazie alla notazione asintotica O :

- $T(n) = 3F_n - 2 \implies T(n) = O(F_n)$
- $T(n) = 2n$ oppure $T(n) = 4n \implies T(n) = O(n)$
- $T(n) = F_n \implies T(n) = O(2^n)$ (poiché $F_n < 2^n$)

7 Approccio Avanzato: Calcolo Matriciale e Potenze Ricorsive

7.1 Possiamo fare meglio di $O(n)$?

Ci poniamo il problema se sia possibile calcolare F_n in tempo **sub-lineare**, ovvero in tempo inferiore ad $O(n)$ (es. in tempo logaritmico $O(\log n)$).

7.2 Proprietà Matriciale dei Numeri di Fibonacci

Proposizione 7.1 (Relazione Matriciale di Fibonacci). Per ogni $n \geq 1$, la n -esima potenza della matrice $M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ è legata ai numeri di Fibonacci dalla seguente identità:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} \quad (11)$$

Dimostrazione per Induzione su n .

1. **Base dell'induzione** ($n = 1$):

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^1 = \begin{pmatrix} F_2 & F_1 \\ F_1 & F_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \quad (12)$$

L'uguaglianza è verificata valendo $F_2 = 1, F_1 = 1, F_0 = 0$ (si pone per convenzione $F_0 = 0$, coerente con $F_2 = F_1 + F_0$).

2. **Ipotesi Induttiva:** Assumiamo vera la proprietà per $n - 1$, ovvero che:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} = \begin{pmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{pmatrix} \quad (13)$$

3. **Passo Induttivo:** Dimostriamo la validità per n :

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} \cdot \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \quad (14)$$

Eseguendo il prodotto riga-per-colonna tra matrici 2×2 :

$$= \begin{pmatrix} F_n \cdot 1 + F_{n-1} \cdot 1 & F_n \cdot 1 + F_{n-1} \cdot 0 \\ F_{n-1} \cdot 1 + F_{n-2} \cdot 1 & F_{n-1} \cdot 1 + F_{n-2} \cdot 0 \end{pmatrix} = \begin{pmatrix} F_n + F_{n-1} & F_n \\ F_{n-1} + F_{n-2} & F_{n-1} \end{pmatrix} \quad (15)$$

Applicando la definizione ricorsiva $F_{n+1} = F_n + F_{n-1}$ e $F_n = F_{n-1} + F_{n-2}$:

$$= \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} \quad (16)$$

La proprietà è formalmente dimostrata per ogni n .

□

7.3 Algoritmo fibonacci5 (Moltiplicazione Sequenziale)

Sfruttando la proprietà matriciale, calcoliamo la potenza M^{n-1} moltiplicando la matrice per se stessa $n - 1$ volte:

```

algoritmo fibonacci5(intero n) → intero
  M ← [[1, 0], [0, 1]] // Matrice Identità 2x2
  for i = 1 to n-1 do
    M ← M * [[1, 1], [1, 0]]
  return M[0][0]

```

Listing 6: Algoritmo fibonacci5

Poiché l'algoritmo esegue $n - 1$ moltiplicazioni di matrici 2×2 (ognuna delle quali costa $O(1)$ tempo), il tempo di esecuzione è ancora $O(n)$. **Guadagno concettuale:** Abbiamo trasformato la determinazione di F_n nel problema del **calcolo di una potenza**.

7.4 Il Principio dell'Elevamento a Potenza Ricorsivo

Per calcolare una generica potenza A^k in tempo logaritmico, si applica la tecnica dell'**Exponentiation by Squaring** (dimezzamento dell'esponente):

$$A^k = \begin{cases} (A^{k/2})^2 & \text{se } k \text{ è pari} \\ A \cdot (A^{\lfloor k/2 \rfloor})^2 & \text{se } k \text{ è dispari} \end{cases} \quad (17)$$

Esempio numerico: Per calcolare 3^8 :

- $3^2 = 9$
- $3^4 = (3^2)^2 = 9^2 = 81$
- $3^8 = (3^4)^2 = 81^2 = 6561$ (effettuate soltanto 3 moltiplicazioni anziché 7).

7.5 Algoritmo fibonacci6

Applichiamo il principio dell'elevamento a potenza ricorsivo alle matrici. Qui M parte dalla matrice base $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ (non dall'identità), perché la procedura calcola M elevata a $n - 1$ a partire da M stessa:

```

algoritmo fibonacci6(intero n) → intero
  M ← [[1, 1], [1, 0]]
  potenzaDiMatrice(M, n-1)
  return M[0][0]

procedura potenzaDiMatrice(matrice M, intero n)
  if (n > 1) then
    potenzaDiMatrice(M, n/2)
    M ← M * M
    if (n è dispari) then
      M ← M * [[1, 1], [1, 0]]

```

Listing 7: Algoritmo fibonacci6

7.6 Analisi della Complessità Temporale di fibonacci6

Nel calcolo del tempo $T(n)$ per la procedura potenzaDiMatrice:

1. All'interno della procedura si spende tempo costante $c = O(1)$ per eseguire le moltiplicazioni tra matrici 2×2 .
2. Viene effettuata **una sola chiamata ricorsiva** sulla metà dell'input ($n/2$).

Otteniamo la seguente equazione di ricorrenza:

$$T(n) = T(n/2) + c \quad (18)$$

Risolviamo mediante il **Metodo dell'Iterazione (Sostituzione)**:

$$\begin{aligned} T(n) &\leq T(n/2) + c \\ &\leq (T(n/4) + c) + c = T(n/4) + 2c \\ &\leq (T(n/8) + c) + 2c = T(n/8) + 3c \\ &\dots \\ &\leq T\left(\frac{n}{2^k}\right) + k \cdot c \end{aligned} \quad (19)$$

L'iterazione si arresta quando la dimensione dell'input scende a 1, ossia quando:

$$\frac{n}{2^k} = 1 \implies 2^k = n \implies k = \log_2 n \quad (20)$$

Sostituendo $k = \log_2 n$ nell'equazione:

$$T(n) \leq T(1) + c \cdot \log_2 n = O(\log_2 n) \quad (21)$$

Risultato: L'algoritmo `fibonacci6` ha un tempo di esecuzione **logaritmico** $O(\log n)$, risultando **esponenzialmente più veloce** degli algoritmi lineari `fibonacci3` e `fibonacci4`.

Occupazione di memoria: La ricorsione genera una pila di chiamate (*call stack*) di profondità pari all'altezza dell'albero ricorsivo, ovvero $k = \log_2 n$. L'occupazione di memoria è pertanto $O(\log n)$.

8 Quadro Sinottico Comparativo di tutti gli Algoritmi

A conclusione del capitolo, sintetizziamo le caratteristiche fondamentali di tutti gli algoritmi analizzati per il calcolo dei numeri di Fibonacci:

Algoritmo	Paradigma / Metodo	Tempo $T(n)$	Spazio $S(n)$	Correttezza
<code>fibonacci1</code>	Formula analitica di Binet	$O(1)^*$	$O(1)$	NO (Err. Floating-Point)
<code>fibonacci2</code>	Ricorsione diretta	$O(2^n)$	$O(n)$	SÌ (Inutilizzabile, lento)
<code>fibonacci3</code>	Prog. Dinamica (Array)	$O(n)$	$O(n)$	SÌ (Molto veloce)
<code>fibonacci4</code>	Prog. Dinamica (Variabili)	$O(n)$	$O(1)$	SÌ (Spazio ottimale)
<code>fibonacci5</code>	Matrice sequenziale	$O(n)$	$O(1)$	SÌ (Iterativo matriciale)
<code>fibonacci6</code>	Potenza matriciale ricorsiva	$O(\log n)$	$O(\log n)$	SÌ (Ottimale in tempo)

Tabella 4: Quadro comparativo finale della complessità e correttezza degli algoritmi per F_n .